

Query Quest

Assignment 1

Neelatmajam Dwivedi

7 June 2026

Part I: Short Answer Questions

Q1. DBMS vs. Traditional File System A traditional file system stores data in isolated files, making it hard to search, secure, and manage relationships between data. A Database Management System (DBMS) organizes data efficiently, enforces relationships, and provides built-in tools for security, backups, and concurrent access.

- **Concrete Scenario:** A banking system handling account balances. If two people try to withdraw money from a joint account at the exact same millisecond using ATMs in different cities, a traditional file system might let both withdrawals go through based on the initial balance, resulting in an overdraft. A DBMS uses concurrency control (ACID properties) to lock the record during the first transaction, ensuring the second transaction waits and only reads the updated balance. This makes the DBMS the clear winner.

Q2. Strong vs. Weak Entities

- **Distinction:** A strong entity has its own primary key and can exist independently. A weak entity does not have a primary key of its own and depends on a strong entity for its existence.
- **Primary Key Formation:** The primary key of a weak entity is formed by combining the primary key of its parent (the strong entity) with its own partial key (often called a discriminator).
- **Identifying Relationship:** The relationship connecting the weak entity to its strong parent is called an “identifying relationship”.

Q3. Modeling a Phone Number

- **Answer:** It should be modeled as a **multivalued attribute**.
- **Justification:** Students can have multiple phone numbers.

Q4. Candidate Key vs. Super Key

- **Difference:** A super key is any combination of columns that uniquely identifies a row in a table. A candidate key is a *minimal* super key, meaning it has no unnecessary columns. If you remove any column from a candidate key, it can no longer uniquely identify a row.
- **Relation:** Student(roll_no, email, name, branch)
- **Candidate Keys:** roll_no, email.
- **Super Keys (that are NOT candidate keys):** {roll_no, name}, {email, branch}.

Q5. Invalid student_id in Enrollment Table

- **Violated Constraint:** The **Foreign Key constraint** (also known as the Referential Integrity constraint) is violated.
- **What it protects against:** It protects against “orphan records” and data inconsistency. It ensures that you cannot assign an enrollment record to a student who does not actually exist in the database.
- **What should happen:** The database should reject the insert operation and return an error.

Q6. Mapping a Many-to-Many (M:N) Relationship

- **Additional Structure:** A new table, commonly called a “junction table” or “associative entity table,” must be created.
- **Primary Key:** Its primary key consists of a combination of the foreign keys referencing the primary keys of the two participating entity tables.
- **Why not a column:** Relational databases operate on the rule of atomicity (First Normal Form). You cannot store a list of multiple related items in a single column without breaking normalization rules, which would cause data redundancy and make querying incredibly difficult.

Part II: Long Answer Question

Part A: Entity and Relationship Identification

- **Entities & Key Attributes:**
 - **Customer:** customer_id (Primary Key)
 - **Restaurant:** restaurant_id (Primary Key)
 - **Order:** order_id (Primary Key)

- **MenuItem:** `item_id` (Primary Key)
- **Relationships:**
 - **Customer places Order (1:N):** A customer places multiple orders over time, but a single order is placed by exactly one customer.
 - **Restaurant receives Order (1:N):** A restaurant processes many orders, but a specific order goes to exactly one restaurant.
 - **Restaurant offers MenuItem (1:N):** A restaurant has many items, but a specific menu item entry belongs to one restaurant.
 - **Order contains MenuItem (M:N):** An order contains multiple items, and the same item can be requested across many orders.

Part B: Attribute Classification

- **Multivalued Attribute:** `dietary_preferences` (for Customer). A customer might be both “Vegan” and “Gluten-Free.”
- **Composite Attribute:** `address` (for Customer or Restaurant). An address naturally breaks down into sub-parts like `street`, `city`, `state`, and `zip_code`.

Part C: Relational Schema Design To resolve the M:N relationship between Order and MenuItem, we create a junction table:

`Order_MenuItem(order_id, item_id, quantity)`

- **Primary Key:** (`order_id`, `item_id`) (A composite primary key).
 - **Foreign Keys:** `order_id` references `Order(order_id)`, and `item_id` references `MenuItem(item_id)`.
 - **Additional Attributes:** `quantity`.
-

Part III: ER Diagram Design

Question 1: Hospital ER Diagram

Question 2: Design Justification

Doctor-Patient Relationship: The Doctor-Patient relationship is many-to-many (M:N) because a patient’s care often requires a team; they can be treated by multiple doctors during their stay. Likewise, a single doctor is responsible for treating multiple patients on their rounds. In a relational database, this creates an associative entity, which we could name `Treatment`. Its composite primary key would consist of the foreign keys from both tables: (`doctor_id`, `patient_id`).

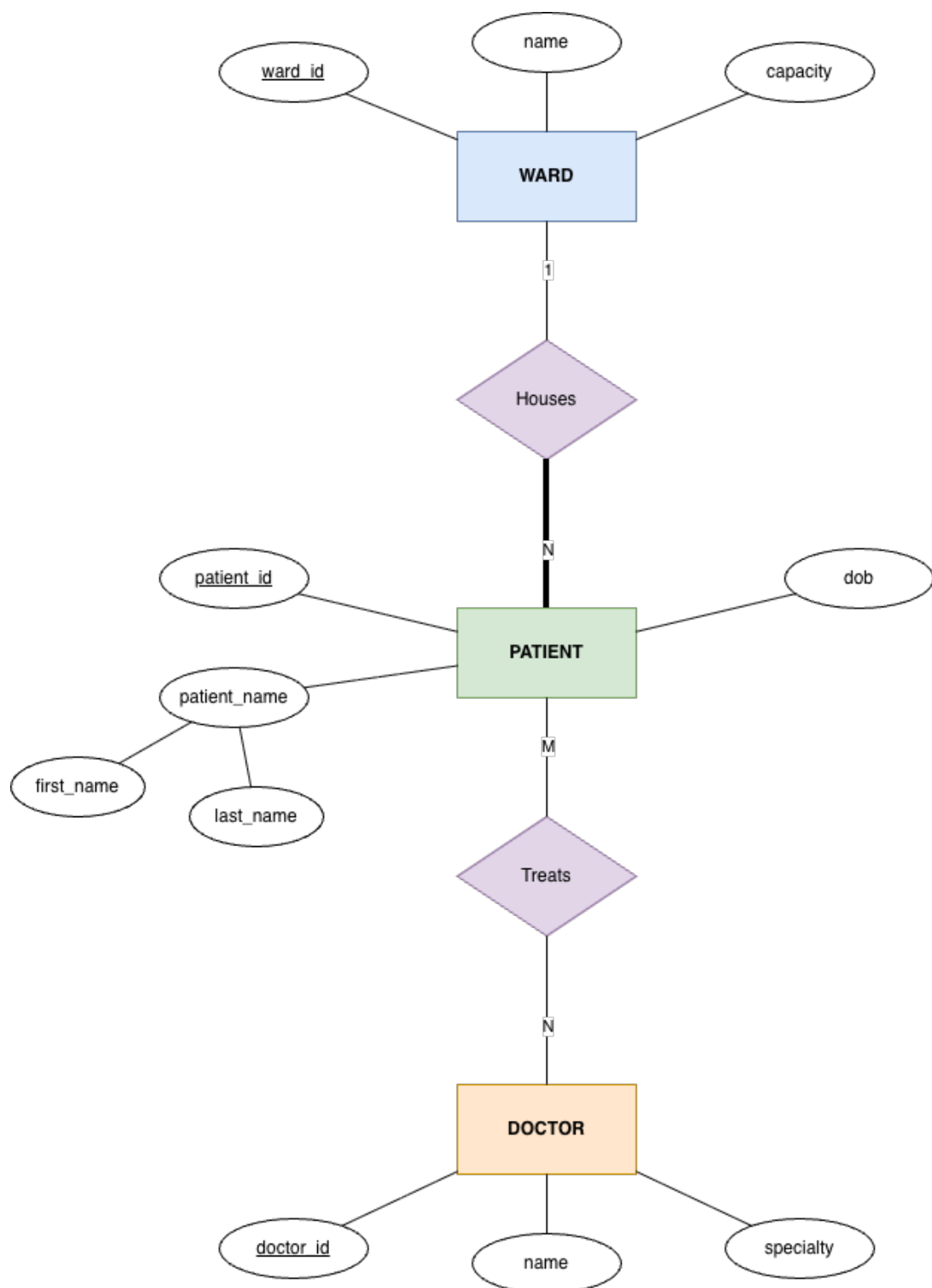


Figure 1: Hospital Entity-Relationship Diagram

Participation Constraints:

1. **Patient to Ward (Houses):** The Patient entity has **total participation**. In a standard hospital system, a patient must be admitted to a ward to officially be tracked as an inpatient; a patient cannot exist in the system without an assigned physical location.
2. **Doctor to Patient (Treats):** The Doctor entity has **partial participation**. A doctor exists in the hospital's staff database even if they currently have zero patients assigned to them (e.g., newly hired, on leave, or strictly researching).

Attribute Design Decision: I modeled the patient's `patient_name` as a **composite attribute** broken down into `first_name` and `last_name`. I made this decision because hospitals frequently need to sort, search, and file patient records alphabetically by last name. Storing the name as a single string makes it computationally difficult to cleanly extract the last name for sorting purposes.